

RV32IMACBFV — Sıfırdan ASIC RISC-V Çekirdeği: Detaylı Yol Haritası

Sentezlenebilir, ASIC hedefli bir **RV32IMACBFV** çekirdeğini sıfırdan inşa etmek için kapsamlı plan. Bu sürüm, ISA fazlarının yanına bir de **mikromimari evrimi** ekliyor: çekirdeği önce **single-cycle**, sonra **5-stage pipeline**, sonra **deep pipeline**, ve (istersen) **superscalar** ve **out-of-order (OoO)** olarak olgunlaştırmak. Her aşamanın alt blokları ve okuma referansları da bölüm bölüm veriliyor.

0. Temel Felsefe

Üç ilke:

- Önce minimal, sonra ekle.** Baştan RV32IMACBFV + OoO yazmak intihardır. Önce çıplak RV32I'yı en basit mikromimariyle (single-cycle) çalıştır; sonra hem ISA özelliklerini (M, A, C, B, F, V) *hem de* mikromimari karmaşıklığını (pipeline derinliği, superscalar, OoO) ayrı eksenler olarak, tek tek, doğrulayarak ekle.
- Doğrulama bir faz değil, sürekli zemin.** Test altyapısı (özellikle bir *golden reference model* — Spike) RTL'den önce kurulur. Simülasyonda yakalanmayan bir bug silikonda 100 kat pahalıya patlar.
- ASIC kaygıları paralel akar.** Sentezlenebilir kod stili birinci günden. Fiziksel tasarım sona doğru yoğunlaşır ama hiç ertelenmez.

İki ayrı eksen olduğunu unutm: - **ISA eksen** — hangi komutları destekliyorsun (I → M → C → B → A → F → V). - **Mikromimari eksen** — o komutları hangi hızda/karmaşıklıkta çalıştırıyorsun (single-cycle → 5-stage → deep → superscalar → OoO).

Bu ikisi büyük ölçüde bağımsız: RV32I'yı hem single-cycle hem OoO yapabilirsin. Pratik strateji: **ISA'yı 5-stage pipeline üzerinde büyüt**, mikromimari yükseltmelerini (deep, superscalar, OoO) ISA olgunlaştıktan sonra ayrı bir yükseltme olarak yap.

Efor göstergesi: [S] küçük · [M] orta · [L] büyük · [XL] devasa.

1. Uzantı Harfleri (IMACBFV)

Harf	Ne ekler	Efor	Bağımlılık
I	Temel 32-bit tamsayı seti (aritmetik, load/store, dallanma)	[M]	Temel
M	Tamsayı çarpma/bölme (MUL, DIV, REM...)	[M]	I
A	Atomikler (LR/SC, AMO)	[M]	I + gerçek bellek arayüzü
C	Sıkıştırılmış 16-bit komutlar	[M]	I (fetch/decode değişir)
B	Bit manipülasyonu (Zba/Zbb/Zbc/Zbs)	[M]	I
F	Tek-duyarlıklı IEEE-754 kayan nokta	[L]	Zicsr (fcsr)
V	Vektör (SIMD; VLEN/SEW/LMUL)	[XL]	I, Zicsr (vtype/vl), çoğunlukla F

Gizli ama zorunlu iki uzantı (tek harfle gösterilmez):

Uzantı	Ne ekler	Efor	Not
Zicsr	CSR komutları (CSRRW/S/C + immediate) + 12-bit CSR adres alanı	[S]	Temel I 'de DEĞİL, ayrı uzantı. F (fcsr), V (vtype/vl) ve M-mode onu zorunlu kılar → fiilen her zaman açık. Faz 1'in parçası.
Zifencei	FENCE.I (komut belleği senkronizasyonu)	[S]	Yine ayrı; I-cache/self-modifying code için gerekir

Önemli: **Zicsr** *unprivileged* spec'te tanımlıdır — yani trap/privileged makinesinden bağımsız, **temel çekirdeğe ait** bir mekanizmadır. Bu yüzden onu Faz 2'ye değil, **Faz 1'e (temel çekirdek)** koyuyoruz. Trap/interrupt makinesi (Faz 2) bu motoru *kullanır*, onu *getirmez*. Tam hedef dizesi: `rv32imacbfv_zicsr_zifencei`.

Bu harfler (ve Zicsr) *unprivileged*; ama gerçek yazılım ve trap/interrupt için ayrıca **Machine-mode** privileged altyapı da gerekir (Faz 2).

2. Paralel Track'ler

- **Track T — Tasarım (RTL):** Fazlar (Faz 0–9) + mikromimari evrimi (Bölüm 3).
- **Track D — Doğrulama:** unit → co-sim → compliance → random → formal → coverage. (Bölüm 13)
- **Track F — Fiziksel/ASIC:** sentez → DFT → floorplan → P&R → STA → signoff. (Bölüm 14)
- **Track P — FPGA prototipi:** her büyük milestone'da gerçek yazılım. (Bölüm 15)

3. MİKROMİMARİ EVRİMİ (Single-cycle → OoO)

Bu, çekirdeğin *hızını ve karmaşıklığını* belirleyen eksen. Aynı ISA'yı giderek daha gelişmiş mikromimarilerde çalıştırırsın. Aşama aşama gidelim; her birinde **ne olduğu**, **gerekli alt bloklar**, **artı/eksi** ve **okuma** var.

3.0 Aşama M0 — Single-cycle [S]

Her komut tek clock'ta biter. İlk ve en basit hedefin.

Ne: Bir clock çevriminde PC → fetch → decode → register oku → ALU → bellek → writeback, hepsi ardışık kombinasyonel yolla. CPI = 1 ama clock periyodu = en yavaş komutun tam yolu (genelde load: PC→IMEM→regfile→ALU→DMEM→regfile). Yani frekans berbat.

Gerekli alt bloklar: - **PC register** (program counter) + PC+4 toplayıcı. - **Instruction memory** (okuma portu). - **Register file** — 2 okuma + 1 yazma portu, x0=0. - **Immediate generator** — komut tipine göre immediate'ı çıkarır (I/S/B/U/J). - **ALU** + ALU control. - **Data memory** (load/store). - **Control unit** — tamamen kombinasyonel decode: opcode/funct → kontrol sinyalleri. - **Mux'lar** — ALU kaynağı, writeback kaynağı, sonraki PC seçimi.

Artı: En basit tasarım ve doğrulama; mükemmel ilk milestone; ders kitaplarıyla birebir. **Eksi:** Çok düşük frekans; gerçek ASIC için uygun değil — ama başlangıç için ideal.

Okuma: Patterson & Hennessy *COD RISC-V Edition* Böl. 4.1–4.4; Harris & Harris *DDCA RISC-V Edition* Böl. 7.3.

3.1 Aşama M1 — Multi-cycle (opsiyonel köprü) [S]

Tarihsel/kavramsal ara adım. Genelde atlanır.

Ne: Her komutu birkaç kısa cycle'a bölüp donanımı (tek ALU/bellek) yeniden kullanmak. CPI > 1 ama clock daha hızlı. Modern pratikte çoğu tasarım single-cycle'dan doğrudan pipeline'a geçer; bu adımı sadece kavramsal olarak bil, dwell etme.

Okuma: Harris & Harris *DDCA* Böl. 7.4.

3.2 Aşama M2 — Klasik 5-stage Pipeline (IF/ID/EX/MEM/WB) [M]

Asıl çalışma atın. ISA'yı bunun üzerinde büyüteceksin.

Ne: Komutları örtüştür — her aşama farklı bir komut üzerinde çalışır. Throughput ~1 komut/cycle ama clock çok daha yüksek (çünkü her aşamanın yolu kısa). Aşamalar: IF (fetch) · ID (decode/register oku) · EX (ALU/dallanma) · MEM (bellek) · WB (writeback).

Yeni alt bloklar (M0'a ek olarak): - **Pipeline registers** — IF/ID, ID/EX, EX/MEM, MEM/WB arasında durumu tutan latch'ler. - **Hazard detection unit** — veri/kontrol tehlikelerini tespit eder. - **Forwarding / bypass unit** — sonucu geç aşamalardan EX'e geri yönlendirip stall'ı önler. - **Stall/flush logic** — bubble ekleme, dallanma sonrasında pipeline flush. - **Branch resolution** — dallanmayı EX'te (ya da daha erken) çöz. - **(Opsiyonel) basit branch predictor** — statik veya küçük BHT/BTB.

Tehlikeler (hazards): - **Structural** — kaynak çakışması (aynı anda iki erişim). Ayrı I/D bellek ile önlenir. - **Data (RAW/WAR/WAW)** — in-order pipeline'da asıl dert **RAW** (bir komut, önceki komutun henüz yazmadığı sonucu okumak ister). Forwarding + gerektiğinde load-use stall'ı ile çözülür. - **Control** — dallanmalar; yanlış tahminde flush.

Artı: Büyük frekans kazancı, forwarding ile CPI hâlâ ~1. Gömülü/eğitim çekirdeklerinin standardı. **Eksi:** Hazard'lar stall üretir; load-use ve branch cezaları var.

Okuma: P&H *COD RISC-V* Böl. 4.5–4.9; Harris & Harris *DDCA* Böl. 7.5; Onur Mutlu pipelining dersleri.

3.3 Aşama M3 — Deep Pipeline (daha çok aşama) [L]

Aşamaları daha da böl → daha yüksek clock.

Ne: Aşamaları alt-aşamalara böl (örn. 8–15+ aşama): fetch'i 2'ye, decode'u 2'ye, EX'i birkaç aşamaya, ayrı register-read/register-write aşamaları, çok-cycle bellek aşamaları... Her aşamanın kritik yolu kısaltıldığı için clock frekansı artar. Klasik örnekler: Pentium 4 (20–31 aşama), ARM Cortex-A serisi (8–13).

Yeni kaygılar / alt bloklar: - **Daha fazla forwarding yolu** — üretim ile tüketim arası mesafe uzar. - **Çok daha iyi branch predictor** — misprediction cezası derinlikle büyür; **gshare / tournament / TAGE** gibi bir predictor + **Return Address Stack (RAS)** + **BTB** şart olur. - **Daha karmaşık hazard/stall mantığı.** - **Uzayan load-use gecikmesi** — küçük bir **store buffer** faydalı olur. - **Clock dağıtımı ve register overhead'i** — her ekstra aşama flip-flop ve clock yükü ekler.

Deep-pipeline dengesi (kritik kavram): Aşama ekledikçe frekans artar ama bir noktadan sonra misprediction/hazard cezaları ve pipeline-register overhead'i frekans kazancını yer. Bir **tatlı nokta** vardır (Hartstein & Puzak, "optimum pipeline depth" makalesi). Sonsuza kadar derinleştirmek geri teper.

Okuma: Hennessy & Patterson *QA* Appendix C (pipelining) ve Böl. 3; Hartstein & Puzak, "The Optimum Pipeline Depth for a Microprocessor," ISCA 2002; Mutlu dersleri; Pentium 4 mikromimari makalesi.

3.4 Aşama M4 — Superscalar (cycle başına birden çok komut) [L]

IPC > 1: aynı anda N komut fetch/decode/issue/execute.

Ne: Genişlik **N** kadar komutu paralel işler. Birden çok ALU, birden çok issue portu. IPC (Instructions Per Cycle) 1'i geçebilir. İki tür var: - **In-order superscalar** (orijinal Pentium, ARM Cortex-A7): komutları program sırasında ama N-geniş issue eder. Basit ama demetin başı stall olunca hepsi bekler. - **Out-of-order superscalar:** N-geniş ve sırasız (bir sonraki aşama — M5).

Yeni alt bloklar: - **Geniş fetch** — cycle başına N komut; hizalı fetch + fetch buffer. - **Çoklu decoder** — N adet. - **Intra-bundle dependency check** — demet içindeki N komut arası hazard kontrolü (in-order superscalar'da). - **Çoklu yürütme birimi** — birden çok ALU, ayrı FP/bellek boru hatları. - **Geniş register file** — daha çok okuma/yazma portu (port baskısı ciddi bir maliyet). - **Geniş bypass network** — N×N forwarding; genişlikle **karesel** büyür.

Artı: IPC > 1. **Eksi:** Dependency-check ve bypass ağı maliyeti genişlikle hızla artar; in-order superscalar hazard'larla sınırlı.

Okuma: Shen & Lipasti *Modern Processor Design* (kitabın tamamı); H&P *QA* Böl. 3; Smith & Sohi, "The Microarchitecture of Superscalar Processors," Proc. IEEE, 1995.

3.5 Aşama M5 — Out-of-Order (OoO) Execution [XL]

Modern yüksek performanslı CPU'ların mimarisi. Devasa bir adım.

Ne: Komutları program sırasında değil, **operandları hazır oldukça** çalıştır; ama **sırayla commit et** (retire). Böylece cache miss'leri ve uzun işlemleri, bağımsız iş bularak gizlersin (latency hiding). Temeli Tomasulo'nun 1967 algoritması; modern OoO = Tomasulo + ROB (precise exception için) + register renaming (physical register file) + speculation.

Anahtar mekanizmalar / alt bloklar: - **Register renaming (RAT + Physical Register File):** sahte bağımlılıkları (WAR/WAW) yok etmek için mimari register'ları daha büyük bir fiziksel register havuzuna eşle. OoO'yu mümkün kılan şey budur. RAT = Register Alias Table (eşleme tablosu). - **Reorder Buffer (ROB):** uçustaki tüm komutları program sırasında izler; sırayla commit ve precise exception sağlar. - **Reservation Stations / Issue Queue:** operand bekleyen komutları tutar; hazır olunca uyandırır (wakeup) ve seçer (select). - **Wakeup/select logic (scheduler):** bir sonuç üretilince ona bağlı komutları uyandır; her cycle hangi hazır komutların issue edileceğini seç. OoO'nun kalbi ve en zor parçası. - **Common Data Bus (CDB) / result broadcast:** sonuçları bekleyen istasyonlara yayınla. - **Load/Store Queue (LSQ):** bellek sıralaması, store-to-load forwarding, memory disambiguation (bir load, aynı adrese önceki bir store'u beklemeli mi, yoksa spekülasyon devam etsin?). - **Branch prediction (kritik):** OoO derin spekülasyon yapar → mükemmel predictor (TAGE, perceptron) + BTB + RAS; misprediction'da RAT'ın checkpoint/rollback ile geri alınması. - **Retirement/commit logic:** sırayla commit, fiziksel register'ları serbest bırak, exception'ları precise biçimde uygula.

Artı: Latency'yi dramatik biçimde gizler, yüksek IPC. **Eksi:** Muazzam karmaşıklık, alan, güç ve doğrulama yükü. BOOM burada yaşıyor. Muhtemelen v1 için fazla — ama bilmek şart.

Okuma: H&P *QA* Böl. 3 (Tomasulo, ROB, speculation); Shen & Lipasti; Tomasulo'nun 1967 orijinal makalesi; Smith & Pleszkun, "Implementing Precise Interrupts in Pipelined Processors," 1988; BOOM (Berkeley Out-of-Order Machine) teknik raporu + Chris Celio'nun MS tezi; Mutlu OoO dersleri.

3.6 Aşama M6 — Diğer İleri Mimariler (genel bakış) [-]

- **VLIW (Very Long Instruction Word):** Paralelliği *derleyici* geniş komut demetlerine yerleştirir; donanım basitleştir (dinamik zamanlama yok). Itanium, birçok DSP. Superscalar'ın zıttı (orada zamanlamayı donanım yapar).
- **SMT (Simultaneous Multithreading):** Bir OoO çekirdekte birden çok thread koşturup boş slotları doldur (Intel Hyper-Threading). OoO backend'i thread'ler paylaşır.
- **Multicore / manycore:** Çok çekirdek; cache coherence (MESI/MOESI), interconnect (ring, mesh), memory consistency

modelleri.

- **Spekülasyon teknikleri:** branch prediction (yukarıda), value prediction, memory dependence prediction, runahead execution, prefetching.
- **Vektör/SIMD:** Veri-seviyesi paralellik (zaten ISA'da V olarak var).
- **Systolic array / dataflow:** ML hızlandırıcıları (TPU gibi), akış-tabanlı mimariler.
- **Güvenlik mikromimarisi:** Spectre/Meltdown — spekülasyonun yan-kanal sızıntıları. OoO spekülasyonu eklediğinde bu tehdit modeli devreye girer.

Okuma: H&P QA (VLIW, multiprocessors, SMT bölümleri); Sorin, Hill & Wood, "A Primer on Memory Consistency and Cache Coherence"; Mutlu ileri dersleri.

4. Alt Blok Sözlüğü (Hızlı Referans)

Hangi mikromimari aşamasının hangi bloğu getirdiği:

Blok	Ne yapar	İlk hangi aşamada
PC + PC-adder	Sonraki komut adresini üretir	M0 (single-cycle)
Register file	Mimari register'lar (x0-x31)	M0
ALU + immediate gen	Aritmetik/mantık + immediate çıkarımı	M0
Control/decode	Opcode → kontrol sinyalleri	M0
Pipeline registers	Aşamalar arası durum	M2 (5-stage)
Hazard unit	Tehlike tespiti (stall)	M2
Forwarding/bypass	Sonucu geri yönlendir	M2
Branch predictor + BTB + RAS	Dallanma tahmini	M2 (basit) → M3 (ciddi)
Store buffer	Store'ları tamponla	M3
Çoklu ALU / issue port	Paralel yürütme	M4 (superscalar)
Register renaming (RAT + PRF)	Sahte bağımlılığı sil	M5 (OoO)
Reorder Buffer (ROB)	Sıralı commit + precise exception	M5
Reservation station / issue queue	Operand bekleyen komutlar	M5
Wakeup/select scheduler	Hazır komutu seç/uyandır	M5
Load/Store Queue (LSQ)	Bellek sıralaması, disambiguation	M5

5. ISA FAZLARI

Bu fazları, mikromimari olarak 5-stage pipeline (M2) üzerinde yürüt. ISA olgunlaşınca istersen deep/superscalar/OoO'ya yükselt.

Faz 0 — Temeller ve Altyapı [M]

RTL yazmadan önce: bir program derleyip Spike'ta koşturabilmek ve boş RTL iskeletini simüle edebilmek.

- **Kararlar:** HDL = SystemVerilog; başlangıç mikromimarisi = M0 single-cycle → hızlıca M2 5-stage; bellek = başta ideal.
- **Toolchain:** RISC-V GCC/LLVM (test derleme); Verilator (sim); **Spike** (golden model); Verible/Verilator lint; SymbiYosys (formal); CI.
- **Golden model:** Spike **lockstep** co-sim — doğrulamanın bel kemiği.
- **Repo + paketler:** `riscv_pkg`, `core_config_pkg` (feature toggle'lar). ✓ başladı.

✓ **Çıkış:** asm derlenip Spike'ta koşuyor; Verilator boş harness'ı derliyor; co-sim iskeleti hazır.

Faz 1 — RV32I + Zicsr Temel Çekirdek [M]

İlk çalışan işlemci. Önce M0 single-cycle ile RV32I'yı doğruyla, sonra aynı ISA'yı M2 5-stage pipeline'a taşı (pipeline register + hazard + forwarding ekleyerek). **CSR motoru (Zicsr) bu çekirdeğin parçasıdır** — sonraki bir faza ertelenmez.

1a — Tamsayı datapath

- **Yapı taşları:** regfile (x0=0), ALU (önce behavioral +, ölç, gerekirse optimize), decode (`riscv_pkg`), hazard + forwarding, branch flush, ideal bellek arayüzü.
- Önce saf RV32I'yı çalıştır (en basit bring-up); sonra Zicsr'ı ekle.

1b — CSR motoru (Zicsr) + temel CSR'lar

- **csr.sv motoru:** 12-bit CSR adres decode (`csr[11:0]`), atomik **oku-değiştir** semantiği (CSRRW = yaz, CSRRS = set-bits, CSRRC = clear-bits + immediate varyantları), ve izin kontrolü (adresin üst bitleri okunur/yazılır ve privilege seviyesini kodlar).
- **Pipeline entegrasyonu:** CSR komutlarının kendi oku/yaz yolu; CSR yazımından sonra okuma için RAW hazard; bazı CSR'ların yan etkileri (side-effect).
- **Her zaman var olan temel CSR'lar:**
 - Makine bilgisi (çoğunlukla salt-okunur sabit): `misal`, `mvendorid`, `marchid`, `mimpid`, `mhartid`.
 - Sayaçlar: `mcycle`, `minstret` (+ RV32'de üst 32-bit: `mcycleh`, `minstreth`); istersen unprivileged gölgeleri `cycle/time/instret` (Zicntr).
 - **misal = uzantı ilanı:** desteklediğin uzantıları burada bit alanında bildirirsin (her harf bir bit: A=0, B=1, C=2, ... I=8, M=12, V=21). Yazılım hangi özelliklerin var olduğunu buradan okur. `IMACBFV` 'yi resmen "ilan ettiğin" yer.
 - **Not (bootstrapping):** Faz 1'de trap makinesi henüz yok; **uygulanmamış bir CSR'a erişimde illegal-instruction trap'i Faz 2'de tamamlanır.** Faz 1'de bu durumu bir sinyal olarak üret, davranışı Faz 2'ye bırak.

Config bağı: CSR motoru (`Zicsr`) daima açık (M-mode ve F onu zorunlu kılar). `core_config_pkg` toggle'ları ise içeriği açıp kapatır: `F_EN=0` → `fcsr` grubu yok, `V_EN=0` → vektör CSR'ları yok.

- **Doğrulama:** directed asm → riscv-arch-test (I + Zicsr) → Spike co-sim → riscv-dv random.

✓ **Çıkış:** RV32I + Zicsr compliance + N cycle random co-sim temiz; CSR oku/yaz/set/clear ve temel CSR'lar Spike ile eşleşiyor. 🕒 **FPGA #1.**

Faz 2 — Privileged Mimari: Trap / Exception / Interrupt [M]

Gerçek yazılım ve A uzantısı için şart. **CSR motoru zaten Faz 1'de var** — bu faz onu *getirmez, kullanır*; üzerine trap makinesini ve trap CSR'larını ekler.

- **Trap CSR'ları** (mevcut motora eklenir): `mstatus`, `mtvec`, `mepc`, `mcause`, `mtval`, `mie`, `mip`, `mscratch`.
- **Trap makinesi:** exception (illegal instruction — **Faz 1'den gelen illegal-CSR sinyali dahil**, misaligned access, ecall/ebreak) + interrupt (timer / software / external).
- **CLINT:** ilk çevre birimi (timer + software interrupt). `mret`, `wfi`.

✓ **Çıkış:** trap'e girip dönüyor; timer interrupt çalışıyor; uygulanmamış CSR erişimi düzgün trap ediyor; trap CSR'ları Spike ile eşleşiyor.

Faz 3 — Tamsayı Uzantıları: M, C, B [M]

3a — M: çarpıcı (behavioral *; FPGA→DSP, ASIC→Booth-Wallace; retiming ile pipeline; MUL/MULH/U/SU tek işaretli çarpıcı + 33-bit genişletme). Bölücü ✓ **yapıldı** (radix-4, 20.022 test; özel durumlar dâhil). Fusion (MULH+MUL, DIV+REM). **3b — C:** decompressor (16→32-bit), fetch hizalama (`IALIGN=16`), `is_compressed` ✓. **3c — B:** Zba/Zbb/Zbc/Zbs — çoğu yeni ALU işlemi + birkaç küçük özel birim (clz/ctz/cpop, clmul).

✓ **Çıkış:** M+C+B compliance geçiyor. 🕒 **FPGA #2.**

Faz 4 — Atomikler (A) + Gerçek Bellek [M]

"Sihirli bellek"i bırak.

- **LR/SC:** reservation set (adres + valid). **AMO:** amoadd/amoswap/amoand/...
- **Sıralama (RVWMO):** `fence`, `fence.i`.
- Gerçek **bus protokolü** burada tanımlanır (Faz 8'de genişler).

✓ **Çıkış:** LR/SC ve AMO'lar Spike ile eşleşiyor.

Faz 5 — Kayan Nokta (F) [L]

İkinci en zor parça.

- **Ayrı FP regfile (f0-f31) + fcsr/frm/fflags'i Faz 1'deki CSR motoruna ekle** (yeni motor kurmuyorsun, mevcut olana register bağılıyorsun). ✓ not edildi.
- **İşlemler:** add/sub, mul, div, sqrt, **FMA**, compare, min/max, FCLASS, convert, FSGNJ, move.
- **IEEE-754:** yuvarlama modları, subnormal, NaN (quiet/signaling), bayraklar (NV/DZ/OF/UF/NX).
- **İnşa mı, reuse mu?** IEEE-754 FPU'yu sıfırdan yazmak devasa; **FPnew/CVFPU** ciddi zaman kazandırır.
- **Bölme/sqrt:** multiplicative (Newton/Goldschmidt, çarpıcıyı reuse) ya da digit-recurrence.
- **Doğrulama:** yalnız Spike yetmez — **SoftFloat/TestFloat** differential + köşe durum fuzz.

✓ **Çıkış:** arch-test F + SoftFloat differential temiz. 🕒 **FPGA #3.**

Faz 6 — Vektör (V) — Büyük Olan [XL]

Efor olarak geri kalan her şeyin toplamı kadar.

- **Vektör regfile:** 32 register × VLEN bit (VLEN seç: 128/256/512). Büyük dizi.
- **Konfigürasyon (CSR):** vtype (SEW/LMUL/tail-mask), vl, vlenb, vstart, vxrm, vxsat — **mevcut CSR motoruna eklenir;** vsetvli/vsetvl/vsetivli komutları bunları ayarlar.
- **Yürütme:** eleman-bazlı ALU, reduction, permütasyon; **lane** organizasyonu (az lane = çok cycle).
- **Load/store:** unit-stride/strided/indexed (gather-scatter)/segment — belleği zorlar.
- **Maskeleme:** v0 mask. **Vektör FP:** F'i reuse eder, fcsr etkileşimi.
- **Mikromimari:** decoupled vektör birimi, skalere karşı hazard izleme; chaining ileri (v1'de atla).
- **Doğrulama:** riscv-dv vektör, arch-test V, Spike co-sim; SAXPY/memcpy/dot kernel'leri.

✓ **Çıkış:** V arch-test + gerçek kernel'ler eşleşiyor. 🕒 **FPGA #4.**

Faz 7 — Bellek Hiyerarşisi (Cache'ler) [L]

- I-cache + D-cache (set/way, replacement, write-back/allocate, coherence: tek çekirdek yok).
- Etkileşimler: A (cache'li atomikler), V (bant genişliği), **fence.i** (I-cache invalidation).
- **TLB/MMU (Sv32):** yalnız S-mode + sanal bellek eklersen — opsiyonel, büyük dal.

Faz 8 — SoC Entegrasyonu + Çevre Birimleri [M]

- **Bus:** AXI4 / TileLink / Wishbone. **Birimler:** PLIC, UART, timer, GPIO, (SPI).
- **Boot:** reset vektörü, boot ROM, program yükleme.

Faz 9 — Özel Komutlar [M]

- Custom opcode alanı (custom-0..3): **riscv_pkg** + decode + özel fonksiyonel birim.
- Directed custom testleri (hazır compliance yok).

13. Track D — Doğrulama (Baştan Sona) [L]

Katmanlar: (1) unit TB (self-checking, bölücü TB'miz gibi) → (2) modül → (3) çekirdek **Spike lockstep co-sim** (beygir) → (4) riscv-arch-test compliance → (5) riscv-dv random → (6) riscv-formal/SymbiYosys formal → (7) coverage kapanışı.

- **FP:** SoftFloat/TestFloat. **CI:** commit→lint+unit+smoke; gecelik→tam compliance+uzun random.
- **UVM vs cocotb:** bu ölçekte **cocotb** (Python) daha pragmatik.

14. Track F — Fiziksel / ASIC [XL]

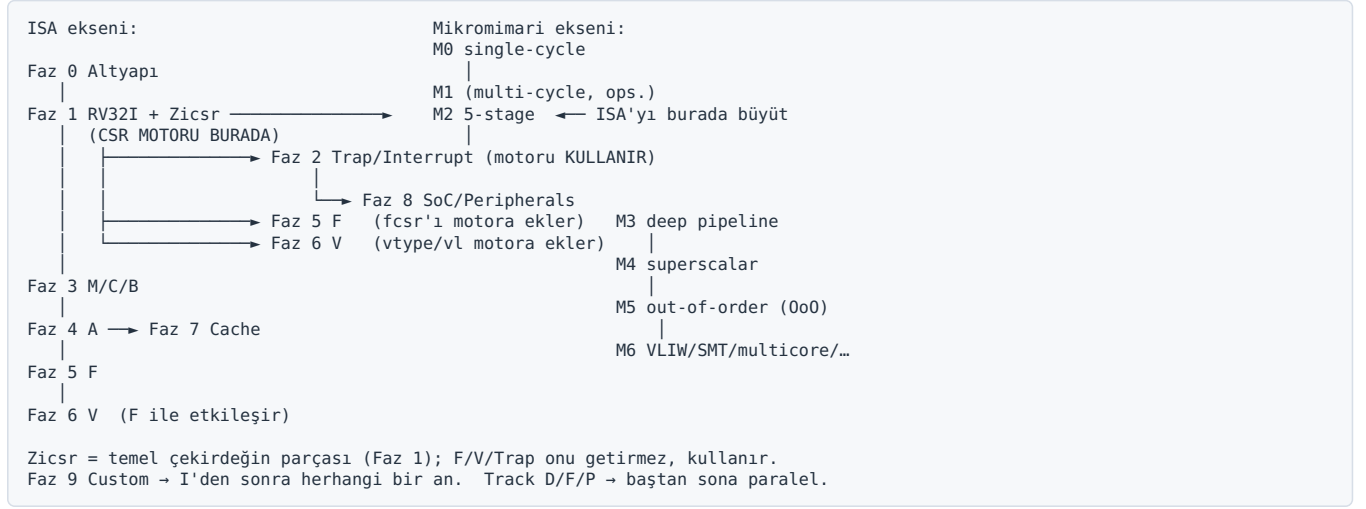
Sentezlenebilir RTL (gün 1) → **logic synthesis** (Yosys açık / DC-Genus ticari; SDC) → **clock/reset** (tek clock, reset senk, CDC) → **DFT** (scan, ATPG; SRAM'ler için MBIST) → **floorplan** (büyük diziler: vektör regfile, cache) → **P&R** (OpenROAD / Innovus-ICC2) → **CTS** → **STA signoff** (setup/hold, PVT köşeleri, MCM) → **power** (clock gating, analiz) → **physical verification** (DRC/LVS/antenna, IR-drop/EM) → **GDSII tapeout**. Ayrıca memory compiler, standart hücre kütüphanesi, I/O pad'ler.

15. Track P — FPGA Prototipi

Her büyük milestone'da FPGA'ye indir (#1 RV32I, #2 M/C/B, #3 F, #4 V). * →DSP, regfile→BRAM. Amaç: mantığı doğrulamak

ve yazılımı hızlı koşturmak (ASIC timing'ini değil).

16. Bağımlılık Grafiği



17. Efor Gerçeklik Kontrolü

- **V ≈ (I+M+A+C+B+F toplamı).** F ikinci en zor. **OoO (M5) tek başına V kadar veya daha büyük.**
- Hızlanmak için doğrulamayı kesme; simülasyonda kaçan bug silikonda 100 kat pahalı.
- Her fazın sonunda çekirdek **çalışır ve doğrulanmış** olsun — yarım özellik biriktirme.
- Mikromimariyi erken zorlama: önce ISA'yı 5-stage üzerinde bitir; deep/superscalar/OoO ayrı ve büyük yükseltmeler.

18. Şu Ana Kadar Ne Yaptık?

- ✓ Repo yapısı + paketler (`riscv_pkg`, `core_config_pkg`) — Faz 0.
- ✓ Toplayıcı stratejisi (behavioral `+`, ölç, optimize) — Faz 1.
- ✓ Çarpıcı stratejisi (behavioral `*`; FPGA-DSP, ASIC-Booth-Wallace) — Faz 3a.
- ✓ Bölücü (radix-4, 20.022 test) — Faz 3a. (Aşama 2 SRT = opsiyonel perf.)

Konum: **Faz 0-1 arası**, mikromimari olarak **M0→M2 geçişinde**, ve Faz 3'ün bölme parçasını erkenden bitirmiş durumda.

19. Hemen Sıradaki 3 Adım

1. **Faz 0'ı bitir:** Spike lockstep co-sim harness'ı.
2. **Faz 1 iskeleti:** önce M0 single-cycle RV32I; sonra **CSR motorunu** (`csr_sv` — **12-bit adres decode + atomik oku/yaz/set/clear + temel CSR'lar: misa, mhartid, sayaçlar**) ekle; ardından M2 5-stage'e taşı.
3. **Bölücüyü bağla:** M fazında `div_r4` 'ü çarpıcıyla `muldiv_unit` 'te birleştirip EX'e bağla.

20. KAYNAKLAR / REFERANSLAR

20.1 Temel / Implementer Kitapları

- **Harris & Harris, *Digital Design and Computer Architecture, RISC-V Edition*** — kapıdan çalışan RISC-V'e; single-cycle → multicycle → pipeline, HDL ile. (Böl. 7) Başlangıç için ideal.
- **Patterson & Hennessy, *Computer Organization and Design, RISC-V Edition*** — single-cycle ve pipelined RISC-V, hazard/forwarding. (Böl. 4) Standart.

20.2 İleri Mikromimari

- **Hennessy & Patterson, *Computer Architecture: A Quantitative Approach* (6. baskı)** — Appendix C (pipelining), Böl. 3 (ILP: superscalar, Tomasulo, ROB, speculation, branch prediction), multiprocessor/SMT/VLIW bölümleri. Bu alanın referansı.
- **Shen & Lipasti, *Modern Processor Design: Fundamentals of Superscalar Processors*** — superscalar/OoO mühendisliğinin kesin kitabı.

20.3 Ücretsiz Dersler

- **Onur Mutlu (ETH Zürich)** — lisans *Digital Design & Computer Architecture* ve graduate *Computer Architecture*; tüm video/slayt/lab "Onur Mutlu Lectures" YouTube kanalında ve safari.ethz.ch'de. Graduate ders pipelining → OoO → superscalar → VLIW → branch prediction akışını kapsar. İleri mimariler için altın kaynak.
- **Berkeley CS152 / CS252 (Krste Asanović)** — RISC-V merkezli; Rocket/BOOM/Chisel ekosistemi.

20.4 Seminal Makaleler

- **R. Tomasulo, "An Efficient Algorithm for Exploiting Multiple Arithmetic Units," IBM J. R&D, 1967** — dinamik zamanlamanın temeli.
- **Smith & Pleszkun, "Implementing Precise Interrupts in Pipelined Processors," 1988** — ROB / precise exception.
- **Smith & Sohi, "The Microarchitecture of Superscalar Processors," Proc. IEEE, 1995** — superscalar genel bakış.
- **Yeh & Patt, two-level adaptive branch prediction (1991-1993)** — modern predictor'ların temeli.
- **Hartstein & Puzak, "The Optimum Pipeline Depth for a Microprocessor," ISCA 2002** — deep-pipeline tatlı noktası.
- **Kessler, "The Alpha 21264 Microprocessor," IEEE Micro, 1999** — klasik bir OoO tasarımı.

20.5 RISC-V Çekirdekleri (Gerçek RTL Okuma)

- **PicoRV32** (minik, okunaklı), **SERV** (bit-serial, en küçük) — eğitim.
- **Ibex** (lowRISC, temiz SystemVerilog, RV32IMC) — sentezlenebilir stil + doğrulama referansı.
- **CV32E40P** (PULP/OpenHW, RV32IMFC + custom DSP) — senin hedefine en yakın örnek.
- **CVA6/Ariane** (app-class), **Rocket** (in-order, Chisel).
- **BOOM (Berkeley Out-of-Order Machine)** — OoO referansı; Chris Celio'nun MS tezi + BOOM teknik raporu.
- **FPnew/CVFPU** — parametrik açık FPU (Faz 5).

20.6 Kayan Nokta Aritmetiği

- **Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic."**
- **Muller et al., *Handbook of Floating-Point Arithmetic*.**
- **Ercegovac & Lang, *Digital Arithmetic*** — bölme/karekök, SRT ve QST türetimi (konuştuğumuz).
- **IEEE 754-2019.**

20.7 Bellek Tutarlılığı / Coherence

- **Sorin, Hill & Wood, *A Primer on Memory Consistency and Cache Coherence*.**

20.8 ASIC / Fiziksel

- **Weste & Harris, *CMOS VLSI Design*.**
- **Açık akış:** OpenLane / OpenROAD + Sky130 dokümantasyonu.

20.9 Doğrulama Araçları

- **Spike** (riscv-isa-sim, golden model), **riscv-arch-test** (compliance), **riscv-dv** (random), **riscv-formal** (formal), **cocotb** (Python TB).

Bu harita canlı bir belgedir. Her fazı ve her mikromimari aşamayı, sırası geldiğinde, istediğin ayrıntı ve yavaşlıkta tek tek açarız.